PROJECT REPORT

ON

Movies Management System

PL/SQL Project

Submitted by:
Shubham Gupta
UID: 26MCI10128

Submitted to:
Prof. Chanchal Sharma
(R303)

Submitted in partial fulfillment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

CHANDIGARH UNIVERSITY

University Institute of Computing
Punjab, India

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"Movies Management System"** has been prepared as an academic project for the subject PL/SQL using Python, Flask, MySQL, HTML, CSS, JavaScript, and related technologies. The project demonstrates web application development, database connectivity, CRUD operations, and the use of MySQL stored procedures.

The application provides an administrator dashboard that enables the administrator to add, update, delete, activate or deactivate, and search movie records. It uses HTML, CSS, and JavaScript for the user interface, Python Flask for backend processing, and MySQL for storing movie information.

This project report is submitted in partial fulfilment of the requirements for the degree of **Master of Computer Applications (MCA)**.

Student Signature: _____ Project Supervisor: _____

Date: _____

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION AND BACKGROUND STUDY 4

1.1 Project Overview 4

1.2 Project Objectives 4

1.3 Technologies Used 4

CHAPTER 2: SYSTEM REQUIREMENTS AND ARCHITECTURE 5

2.1 Main Modules and Features 5

2.2 Project Architecture and Workflow 5

2.3 Database Design 6

CHAPTER 3: IMPLEMENTATION AND DATABASE OPERATIONS 7

3.1 MySQL Stored Procedures 7

3.2 Important Python Libraries 7

3.3 Project Folder Structure and File Roles 7

CHAPTER 4: SETUP, RESULTS AND CONCLUSION 8

4.1 How to Run the Project 8

4.2 Advantages of the Project 9

4.3 Project Limitations 9

4.4 Future Enhancements 9

4.5 Conclusion 10

REFERENCES 10

CHAPTER 1: INTRODUCTION AND BACKGROUND STUDY

1.1 Project Overview

The Movies Management System is a web-based application developed to manage movie information through an administrator dashboard. The project focuses only on the **Admin Module**, in which the administrator can add new movies, update existing movie details, delete movies, activate or deactivate movies, and search for movies.

The application uses Python Flask for the backend, HTML, CSS, and JavaScript for the frontend, and MySQL for database management. The main purpose of this project is to demonstrate how a web application interacts with a relational database and how database operations can be managed using SQL and stored procedures.

As a project for the subject PL/SQL, it demonstrates the use of MySQL stored procedures for adding, updating, deleting, and changing the status of movie records, while the Flask application provides a simple graphical interface for these operations.

1.2 Project Objectives

The primary goals of the Movies Management System are listed below:

1. To develop a simple web-based movie management system.
2. To provide an admin dashboard for managing movie records.
3. To perform CRUD operations on movie information.
4. To store and retrieve movie details using MySQL.
5. To implement MySQL stored procedures.
6. To provide an easy-to-use graphical interface for administrators.

1.3 Technologies Used

The project combines a Python backend, a web-based frontend, and a relational database. The tools and libraries used are listed in Table 1.1.

Table 1.1: Technologies used in the project

Technology	Purpose
Python	Main programming language
Flask	Python web framework
HTML	Structure of web pages
CSS	Styling and layout
JavaScript	Frontend interactions
MySQL	Relational database
MySQL Workbench	Database creation and management
mysql-connector-python	Connects Python with MySQL
python-dotenv	Reads database credentials from the ".env" file

CHAPTER 2: SYSTEM REQUIREMENTS AND ARCHITECTURE

2.1 Main Modules and Features

The admin module is divided into six functional parts: the dashboard, adding, updating, deleting and searching movies, and changing the status of a movie.

2.1.1 Admin Dashboard

The dashboard displays an overview of movie records. Its features are:

- Total number of movies.
- Number of active movies.
- Number of inactive movies.
- List of available movies.

2.1.2 Add Movie

The administrator can add a new movie by entering the following details:

- Movie title
- Genre
- Ticket price
- Duration
- Language

After submission, the movie details are saved in the MySQL database.

2.1.3 Update Movie

The administrator can edit existing movie information. For example, the admin can change the movie title, genre, price, duration, or language.

2.1.4 Delete Movie

The administrator can remove a movie record from the database.

2.1.5 Search Movie

The administrator can search for movies using their title or genre.

2.1.6 Activate and Deactivate Movie

The administrator can change the status of a movie:

- **Active:** the movie is available.
- **Inactive:** the movie is not currently available.

The record remains in the database even when it is deactivated.

2.2 Project Architecture and Workflow

The application follows a simple three-layer architecture consisting of the presentation layer, the application layer, and the database layer (Figure 2.1 and Table 2.1).

Table 2.1: Layers of the project architecture

Layer	Technologies	Responsibility

Presentation Layer	HTML, CSS, JavaScript	Displays the admin dashboard and forms.
Application Layer	Python, Flask	Receives requests from the frontend, processes the data, and communicates with MySQL.
Database Layer	MySQL	Stores movie records and executes SQL queries and stored procedures.

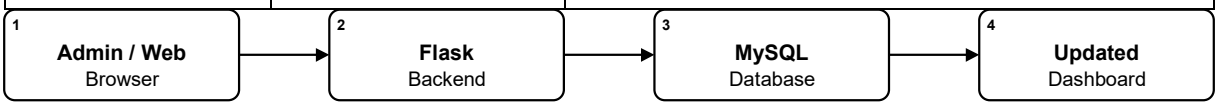


Figure 2.1: Request flow between the administrator, Flask, and MySQL

2.2.1 How the Application Works

1. The administrator opens the web application in a browser.
2. Flask loads the dashboard.
3. Flask connects to the MySQL database.
4. Movie records are retrieved from the database.
5. The dashboard displays the movie information.
6. When the admin adds, updates, or deletes a movie, Flask processes the request.
7. The corresponding database operation is executed.
8. The updated information is displayed on the dashboard.

2.3 Database Design

The project uses a MySQL database named `movie_ticket_db` with a single table named `movies`. The movies table stores all movie-related information, as shown in Table 2.2.

Table 2.2: Structure of the movies table

Column	Description
`id`	Unique movie ID and primary key
`title`	Name of the movie
`genre`	Movie category
`price`	Ticket price
`duration`	Movie duration in minutes
`language`	Movie language
`status`	Active or inactive
`created_at`	Date and time when the record was created

2.3.1 What is a Primary Key?

A primary key is a column that uniquely identifies each record in a table. In this project, `id` is the primary key of the movies table.

2.3.2 What is CRUD?

CRUD represents four basic database operations:

- **Create:** add a new movie.

- **Read:** display movie records.
- **Update:** modify movie details.
- **Delete:** remove a movie record.

These operations form the core functionality of the admin module.

CHAPTER 3: IMPLEMENTATION AND DATABASE OPERATIONS

3.1 MySQL Stored Procedures

The project uses stored procedures to perform database operations. A stored procedure is a group of SQL statements stored inside the database that can be executed whenever required. The stored procedures used in the project are listed in Table 3.1.

Table 3.1: Stored procedures used in the project

Stored Procedure	Purpose
`sp_add_movie`	Adds a new movie
`sp_update_movie`	Updates movie details
`sp_delete_movie`	Deletes a movie
`sp_toggle_movie_status`	Changes movie status

3.1.1 Advantages of Stored Procedures

1. SQL logic can be reused.
2. Database operations can be organized.
3. Application code becomes cleaner.
4. Database operations can be managed centrally.

3.2 Important Python Libraries

Three Python packages are central to the project:

- **Flask:** a lightweight Python web framework used to develop web applications. It handles routes, requests, responses, and HTML templates.
- **mysql-connector-python:** a Python package that allows Python applications to connect with MySQL databases.
- **python-dotenv:** loads environment variables from a ".env" file. The ".env" file stores configuration information such as database username, password, and database name.

3.3 Project Folder Structure and File Roles

The project is organized into separate folders for SQL scripts, HTML templates, and static files, as shown in Listing 3.1.

Listing 3.1: Project folder layout

```
Movie_Ticket_Booking_Admin_MySQL/
|-- app.py
|-- .env
|-- .env.example
|-- requirements.txt
|-- README.md
|-- sql/
```

```
|   |-- schema.sql
|   `-- procedures.sql
|-- templates/
|   |-- index.html
|   `-- error.html
`-- static/
    |-- style.css
    `-- app.js
```

Table 3.2 describes the purpose of the important files.

Table 3.2: Important files of the project and their purpose

File	Purpose
app.py	Main Flask application
.env	Database configuration
requirements.txt	Required Python packages
schema.sql	Creates database and table
procedures.sql	Creates stored procedures
index.html	Admin dashboard
error.html	Displays connection errors
style.css	Dashboard styling
app.js	Frontend interactions

CHAPTER 4: SETUP, RESULTS AND CONCLUSION

4.1 How to Run the Project

The project can be set up and started on a local machine by following the steps below.

Step 1: Open the project folder

Open PowerShell inside the project directory.

Step 2: Install dependencies

Listing 4.1: Installing the required packages

```
py -m pip install -r requirements.txt
```

Step 3: Create the database

Open MySQL Workbench and execute the following statement:

Listing 4.2: Creating the database

```
CREATE DATABASE movie_ticket_db;
```

Step 4: Execute SQL files

Run these files in MySQL Workbench, in this order:

1. `sql/schema.sql`
2. `sql/procedures.sql`

Step 5: Configure the .env file

Listing 4.3: Contents of the .env file

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=movie_ticket_db
FLASK_SECRET_KEY=your_secret_key
```

Replace `your_mysql_password` with your actual MySQL password.

Step 6: Start the application

Listing 4.4: Starting the Flask application

```
py app.py
```

Step 7: Open the application

Open the browser and visit `http://127.0.0.1:5000/`

4.2 Advantages of the Project

1. Simple and user-friendly admin interface.
2. Centralized movie management.
3. Easy movie searching and updating.
4. Organized database structure.
5. Use of stored procedures.
6. Reduces manual database management work.

4.3 Project Limitations

The current version of the project has the following limitations:

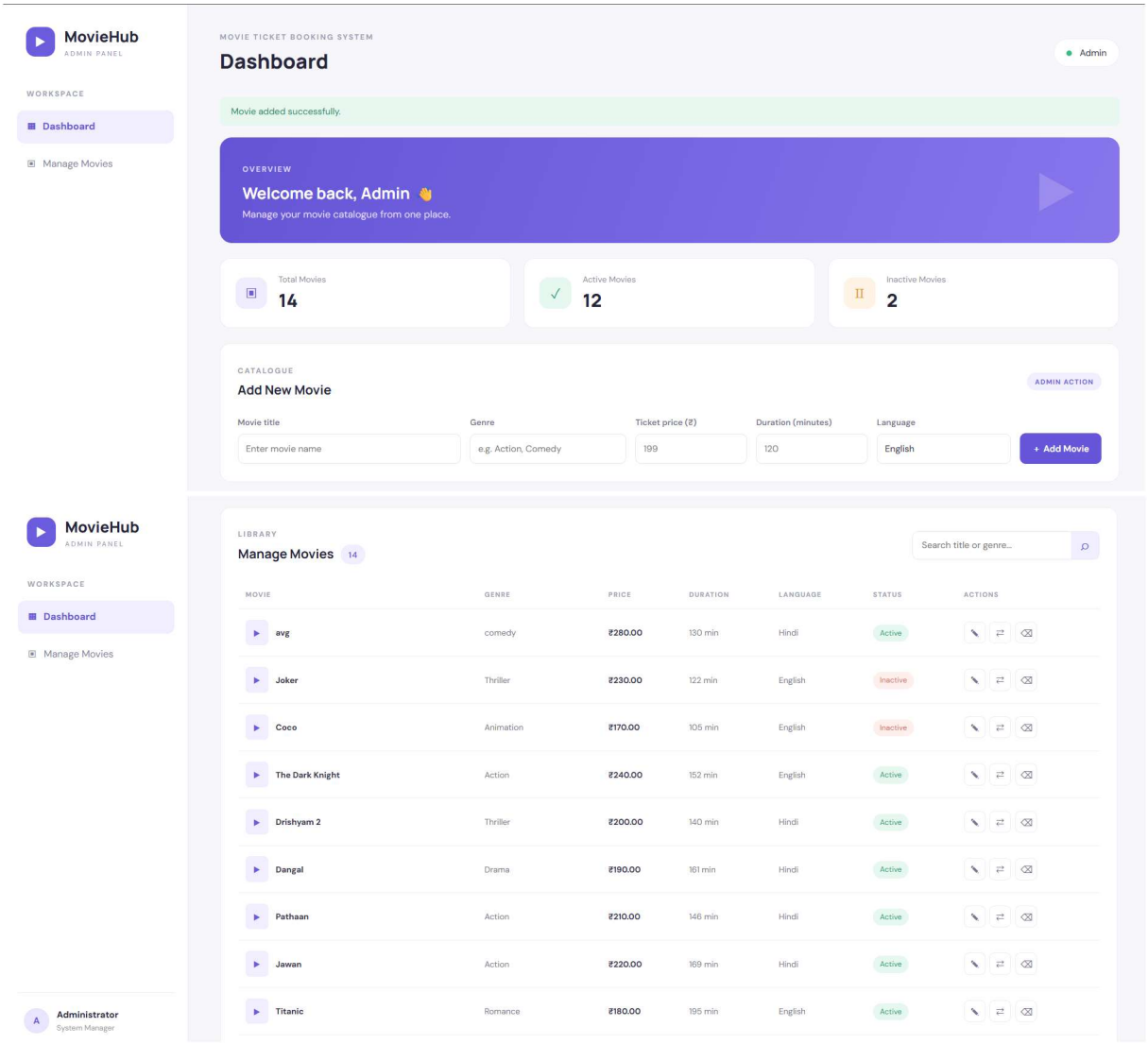
1. The current project focuses only on the admin module.
2. Customer registration and login are not included.
3. Online ticket booking is not implemented.
4. Payment gateway integration is not included.
5. Seat selection and reservation are not implemented.

4.4 Future Enhancements

The project can be extended by adding:

1. Customer registration and login.
2. Movie ticket booking.
3. Seat selection.
4. Online payment integration.
5. Booking history.
6. Admin authentication and role-based access.
7. Reports of movie bookings and revenue.

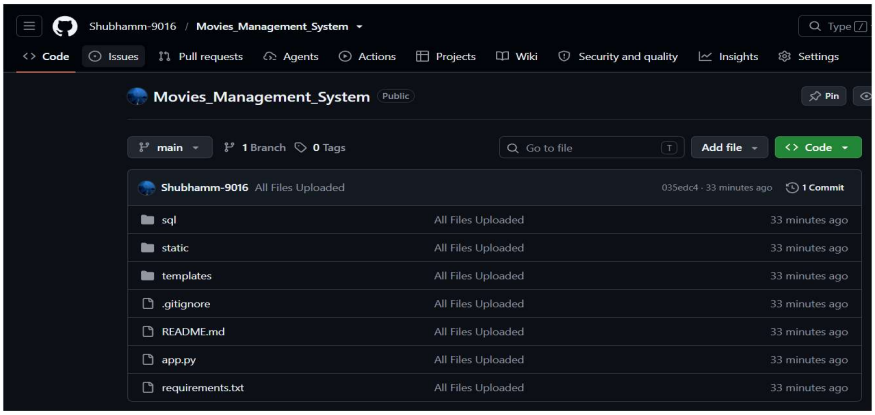
4.5 Output



4.6 Git & Github:

Github link : https://github.com/Shubhamm-9016/Movies_Management_System.git

Github File Uploaded:



4.6 Conclusion

The Movies Management System is a database-oriented web application developed using Python Flask and MySQL.

It provides an administrator with the ability to manage movie records through a graphical interface. The project demonstrates CRUD operations, database connectivity, SQL queries, and stored procedures.

This project provides a foundation for developing a complete movie ticket booking application in the future.

REFERENCES

1. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
3. Pallets Projects. *Flask Documentation.* https://flask.palletsprojects.com/
4. Oracle. *MySQL 8.0 Reference Manual.* https://dev.mysql.com/doc/
5. Oracle. *MySQL Connector/Python Developer Guide.* https://dev.mysql.com/doc/connector-python/en/
6. python-dotenv. *python-dotenv Documentation.* https://pypi.org/project/python-dotenv/